

[INSERTAR LOGO DEL INSTITUTO]

INFORME DE EVALUACIÓN — UNIDAD 3

Desarrollo de la capa Frontend de un Gestor Documental

Escuela Básica G-733 Chorombo Bajo, comuna de María Pinto

Antecedente	Detalle
Asignatura	Desarrollo Frontend
Unidad evaluada	Unidad 3 — Componentes, interactividad y consumo de servicios
Estudiante	Luis Orellana Cabrera
Modalidad	Trabajo individual
Docente	[COMPLETAR]
Fecha de entrega	[COMPLETAR]
Repositorio	github.com/Luisorellanacabrera/gestor-documental-frontend

Cliente: Escuela Básica G-733 Chorombo Bajo — Contraparte: Patricia Gajardo Ordenes, Directora

Índice

Índice	2
1. Introducción	3
2. Propuesta de componentes · Indicador 1	3
3. Resolución de un problema técnico · Indicador 2.....	4
4. Accesibilidad y usabilidad · Indicador 3	4
5. Integración y despliegue · Indicador 4	5
6. Buenas prácticas aplicadas · Indicador 5	6
7. Control de versiones · Indicador 6.....	6
8. Diseño de la interfaz · Indicador 7	6
9. Estrategias de optimización · Indicador 8	7
10. Integración del framework · Indicador 9.....	7
11. Consumo de endpoints · Indicador 10	7
12. Estrategias de seguridad · Indicador 11.....	8
13. Documentación y difusión · Indicador 12	8
14. Retrospectiva y mejora continua · Indicador 13.....	9
15. Conclusiones	9
16. Declaración de uso de Inteligencia Artificial	10
17. Referencias	10

Para actualizar el índice en Word: clic sobre la tabla y presionar F9.

1. Introducción

La Escuela Básica G-733 Chorombo Bajo, de María Pinto, atiende a 208 estudiantes y administra de forma manual su documentación institucional: memos, oficios, citaciones y acuerdos de apoderados, documentos de reuniones comunales y permisos administrativos. La directora planteó centralizar esa información en una plataforma web, señalando que la falta de recursos obliga a optimizar los medios disponibles: no hay infraestructura previa y el único equipo es un notebook de portería con Core i5, 8 GB de RAM y 128 GB de disco.

Este informe documenta la capa Frontend del Gestor Documental, que permite visualizar, buscar, filtrar, registrar, modificar y eliminar documentos consumiendo la API de Desarrollo Backend. Esas restricciones se tradujeron en decisiones concretas: no requiere instalación ni compilación, no depende de gestores de paquetes y sirve Bootstrap localmente, de modo que funcione sin internet. Cada capítulo corresponde a un indicador de la unidad.

Tecnología	Uso
HTML5 semántico	Estructura de la interfaz y base de la accesibilidad
Bootstrap 5.1.1 local	Rejilla, tabla, ventanas modales, avisos y diseño adaptable
JavaScript ES Modules	Componentes, manejo de estado y eventos, sin compilación
Fetch API	Consumo de los endpoints del Backend

Ejecución: con el Backend levantado, la aplicación se sirve con `python3 -m http.server 5500` y queda en `http://127.0.0.1:5500`. Debe abrirse mediante un servidor web y no con doble clic, porque el navegador bloquea los módulos de JavaScript bajo el protocolo `file://`.

2. Propuesta de componentes · Indicador 1

Antes de programar se definió la estructura de componentes. El criterio: cada componente hace una sola cosa y no conoce el funcionamiento interno de los demás; la comunicación pasa por `app.js`, único módulo que los conoce a todos.

Componente	Responsabilidad	Información que recibe	Relación
Navegación	Encabezado, estado de conexión y acción principal	—	Pide a <code>app.js</code> abrir el formulario
<code>resumen.js</code>	Cuántos documentos hay de cada tipo	Estadísticas de la API	Al pulsar una tarjeta, <code>app.js</code> filtra por ese tipo
<code>barraFiltros.js</code>	Recoger texto, tipo y rango de fechas	Catálogo de tipos y función de cambio	Entrega los filtros a <code>app.js</code>
<code>listaDocumentos.js</code>	Dibujar tabla y paginación; avisar la acción pulsada	Documentos, paginación y funciones Ver, Editar, Eliminar	Abren detalle, formulario o confirmación
<code>detalleDocumento.js</code>	Mostrar toda la información de un documento	El objeto documento	Se abre desde el listado; encadena con el formulario
<code>formularioDocumento.js</code>	Capturar y validar datos al crear y editar	Catálogo de tipos y, al editar, el documento	Devuelve datos a <code>app.js</code> y muestra los errores de la API
<code>confirmarEliminar.js</code>	Pedir confirmación antes de borrar	El documento a eliminar	Confirma a <code>app.js</code> , que ejecuta el borrado
<code>mensajes.js</code>	Estados de carga, vacío, error y avisos	Textos	Lo usan todos los flujos de <code>app.js</code>

Componente	Responsabilidad	Información que recibe	Relación
utilidades.js	Escapar HTML, formatear fechas, retrasar la búsqueda	—	Lo usan casi todos los componentes

Reutilización: el mismo formulario crea y edita, evitando dos formularios casi idénticos; mensajes.js centraliza la retroalimentación para que se vea igual en toda la aplicación; y escaparHtml() vive en un solo lugar, de modo que la seguridad no dependa de recordarla caso por caso.

3. Resolución de un problema técnico · Indicador 2

Aspecto	Detalle
Problema	Desde la ventana de detalle, el botón «Editar» debía cerrarla y abrir el formulario. En algunas ejecuciones el formulario no se abría y el botón parecía no responder. El fallo no era constante, lo que dificultaba identificar la causa.
Causa detectada	Dos ventanas modales no pueden estar abiertas a la vez. El código cerraba la ventana y DESPUÉS registraba el escuchador del evento de cierre. Bootstrap demora ~300 ms en la animación, así que alcanzaba a registrarse. Pero si el cierre es instantáneo —lo que ocurre con «reducir animaciones» del sistema activado— el evento se dispara antes de que el escuchador exista, se pierde y el formulario no se abre nunca. Se reprodujo quitando la clase fade, que produce ese mismo efecto.
Alternativas	(1) Esperar con un temporizador fijo de 300 ms: descartada, fija a mano un tiempo que depende del equipo y en un computador lento vuelve a fallar. (2) Una sola ventana que cambie de contenido: evita el problema pero mezcla detalle y formulario, perdiendo la separación de responsabilidades. (3) Registrar el escuchador antes de cerrar: elegida, no depende de tiempos y mantiene los componentes separados.
Solución	Se invirtieron las dos líneas: primero addEventListener con { once: true }, después hide(). Así el escuchador existe sin importar cuánto demore el cierre.
Resultado	Verificado con animaciones desactivadas: el detalle se cierra, el formulario se abre con los datos cargados y queda 1 capa de fondo. Sin la solución quedaban 2 capas y, al cerrar, 1 pegada con el desplazamiento bloqueado, obligando a recargar.
Aprendizaje	Un error que solo aparece con una opción de accesibilidad activada es especialmente riesgoso: en el equipo de desarrollo nunca se ve y deja sin uso a la persona afectada. Se incorporó como práctica registrar el escuchador antes de disparar la acción que emite el evento, y probar con las animaciones desactivadas. La solución conserva la mantenibilidad: detalleDocumento.js sigue sin saber qué es un formulario.

4. Accesibilidad y usabilidad · Indicador 3

Criterio WCAG 2.1	Aplicación en la solución
1.3.1 Información y relaciones	HTML semántico (header, nav, main, section, footer), tabla con th scope y caption, fieldset con legend
1.4.1 y 1.4.3 Color y contraste	El estado de la conexión no se comunica solo con color: junto al punto siempre hay texto. Todos los textos superan 4.5:1, medido y corregido (ver más abajo)
2.1.1, 2.4.1, 2.4.3 y 2.4.7 Teclado, foco y navegación	Toda la aplicación se opera con teclado; el orden de tabulación sigue el orden visual; al abrir una ventana el foco entra al primer campo; enlace «Saltar al contenido» como primer elemento tabulable y contorno de foco de 3 px en todo elemento enfocable
3.1.1 y 3.3.x Idioma y errores	Idioma declarado; cada campo con su etiqueta; los errores describen qué corregir y aparecen junto al campo y en un resumen que recibe el foco
2.3.3 y 4.1.3 Animación y mensajes	Se respeta prefers-reduced-motion; carga, vacío y error dentro de una región aria-live, y avisos con role alert

Prueba realizada. Se ejecutó un guion sobre la aplicación en funcionamiento que calcula la relación de contraste de cada texto según la fórmula de WCAG 2.1 y verifica etiquetas, encabezados, nombres accesibles y regiones dinámicas. Detectó dos incumplimientos reales, ambos corregidos:

Criterio	Problema identificado	Mejora implementada	Resultado
1.4.3 Contraste	El texto descriptivo usaba la clase text-secondary de Bootstrap: 4.37:1 sobre el fondo, bajo el mínimo de 4.5:1	Se redefinió a un gris más oscuro sin alterar el aspecto	5.54:1, cumple
1.4.1 Color	Todos los distintivos de tipo se veían del mismo gris: la regla general de CSS tenía más especificidad que las de cada tipo y pisaba sus colores	Los colores se movieron a reglas de igual especificidad	Cada tipo se distingue, entre 7.05:1 y 8.91:1

El resto no arrojó incumplimientos: ningún control sin etiqueta, ningún botón sin nombre accesible, un solo encabezado de primer nivel, sin saltos de jerarquía y con el idioma declarado.

Revisión de usabilidad. Emulando una pantalla de 375 px se detectó que el botón «Nuevo documento» quedaba dentro del menú desplegable: se sacó de ahí y quedó siempre visible, abreviado en pantallas angostas. El formulario ocupaba 737 px de alto contra 720 px visibles, dejando los botones fuera de la vista: se aplicó desplazamiento interno a la ventana.

[PENDIENTE OBLIGATORIO: la evaluación exige además una revisión con al menos un usuario de prueba o persona externa. Las mejoras anteriores provienen de la revisión técnica propia, no de una prueba con usuario real. Siga el guion del README (sección 8) y registre aquí participante, tarea, dificultad observada y mejora implementada.]

5. Integración y despliegue · Indicador 4

Los nueve componentes se integraron hasta obtener una aplicación funcional, verificada contra el Backend real. La integración es multidisciplinaria porque cruza dos asignaturas: el Frontend consume la API construida en Desarrollo Backend, que corre en otro proceso y otro puerto. Con ambos servidores levantados se ejecutó el ciclo completo, comprobando en el panel de red que cada acción disparara la petición correcta y que la respuesta se reflejara en pantalla:

Acción	Petición	Resp.	Efecto en pantalla
Carga inicial	GET /salud, /tipos-documento, /documentos, /estadísticas	200	«API conectada», selectores llenos, tabla y tarjetas dibujadas
Crear	POST /documentos	201	Aviso de éxito, el total sube y la fila aparece
Buscar	GET /documentos?buscar=...	200	El contador refleja las coincidencias
Ver y editar	GET /documentos/{id} y PUT /documentos/{id}	200	Detalle con todos los campos; dato actualizado en la tabla
Eliminar	DELETE /documentos/{id}	200	Aviso de éxito, el total baja y la fila desaparece

Preflight CORS. Al correr en puertos distintos, el navegador envía una petición OPTIONS previa en cada llamada con cabecera Content-Type; se verificó que el Backend responde 200. Es el tipo de detalle que solo aparece al integrar dos capas desarrolladas por separado.

Configuración. Todas las variables están centralizadas en js/config.js: URL_API con la dirección del Backend (única línea a cambiar para apuntar a otro servidor), DOCUMENTOS_POR_PAGINA en 10, ESPERA_BUSQUEDA en 400 milisegundos y EXTENSIONES_PERMITIDAS con las extensiones aceptadas, que coinciden con las del Backend.

Despliegue. La aplicación son archivos estáticos: se despliega copiando la carpeta a cualquier servidor web (Apache, Nginx, GitHub Pages o Netlify) y ajustando URL_API. No requiere compilación. El procedimiento reproducible está en el README del repositorio.

6. Buenas prácticas aplicadas · Indicador 5

Buena práctica	Propósito y aplicación	Elemento medible	Ubicación
1. Nomenclatura descriptiva	Entender el código sin ejecutarlo; nombres coincidentes con el Backend	renderizarLista, abrirParaEditar	Todo el proyecto
2. Separación por capas	Cambiar una parte sin romper otras; solo la capa de API llama a fetch	0 fetch fuera de documentosApi.js	js/api/, js/componentes/
3. Estructura de carpetas	Encontrar rápido lo que se busca	css, js/api, js/componentes, vendor, docs	Raíz
4. Reutilización	No escribir dos veces lo mismo; un formulario para crear y editar	1 formulario, 2 operaciones	formularioDocumento.js
5. Estado en un solo lugar	Evitar que dos partes tengan datos distintos	0 variables globales	app.js
6. Validación previa	Avisar de inmediato; mismas reglas del Backend	5 reglas validadas	validar()
7. Errores sin detalle técnico	Que el usuario entienda qué pasó	0 trazas expuestas	documentosApi.js
8. Comentarios del porqué	Que quien retome entienda las decisiones, no la sintaxis	Cada decisión no evidente justificada	js/
9. Configuración centralizada	Cambiar de servidor sin buscar valores dispersos	Mover la app exige cambiar 1 línea	config.js
10. HTML semántico	Comprensible para lectores de pantalla sin parches ARIA	1 h1, 0 saltos, 13 th con scope	index.html
11. Dependencias locales	Funcionar sin internet y sin depender de un servicio externo	0 peticiones a dominios externos	vendor/bootstrap/

7. Control de versiones · Indicador 6

Proyecto individual, versionado con Git y publicado en un repositorio público: github.com/Luisorellanacabrera/gestor-documental-frontend. Los mensajes de commit explican qué cambió y por qué, nunca «cambios» o «avance»; cada commit tiene alcance acotado y deja la aplicación funcionando; y el orden del historial refleja cómo se construyó. La evaluación pide usar ramas «cuando sean pertinentes»: en un desarrollo individual y secuencial, sin trabajo paralelo ni conflictos que resolver, una rama por funcionalidad habría sido una formalidad sin función, por lo que la evidencia de este indicador es la trazabilidad del historial y no la coordinación entre integrantes.

[**COMPLETAR:** *agregar una captura del historial de commits del repositorio.*]

8. Diseño de la interfaz · Indicador 7

La interfaz se diseñó pensando en quién la usará: un equipo directivo sin perfil técnico que hoy trabaja en papel. Por eso el vocabulario es el de la escuela —memos, oficios, citas— y no el del sistema. El listado muestra las cinco columnas exigidas: título, tipo, fecha, descripción y acciones; si el documento tiene archivo, junto al título aparece un distintivo con su nombre.

Operaciones y retroalimentación. La tabla, paginada de a diez, permite visualizar; el botón «Ver» abre el detalle completo. Crear y editar comparten formulario, con el botón siempre visible en la barra; mientras guarda, el botón se bloquea y muestra «Guardando...», evitando el doble envío, y al terminar aparece un aviso verde. Eliminar exige confirmación mostrando el título, con el foco en «Cancelar». La

búsqueda por título y descripción y los filtros por tipo y por rango de fechas actualizan un contador de resultados, con mensaje distinto si no hay coincidencias o si el listado está vacío. Ante datos inválidos se muestra un resumen que recibe el foco más el mensaje bajo cada campo; si el servidor no responde, un mensaje en lenguaje común con botón de reintentar.

Diseño adaptable. En celular las tarjetas se reorganizan en dos columnas, la columna de descripción se oculta para que la tabla siga legible —el dato permanece en el detalle— y la acción principal se abrevia sin perder visibilidad. Verificado en 375, 768 y 1280 píxeles de ancho.

[**COMPLETAR:** insertar capturas en escritorio y celular, numeradas según APA.]

9. Estrategias de optimización · Indicador 8

Optimización	Situación inicial	Qué se aplicó	Beneficio
1. Búsqueda con pausa	Se recargaba en cada tecla: «apoderados» generaba 10 peticiones, 9 obsoletas	retrasar() espera 400 ms sin escritura antes de consultar	Medido: 1 sola petición. Reducción del 90 %
2. Catálogo en memoria	Se pedía en cada apertura del formulario, para datos que cambian una vez al año	Se guarda el resultado la primera vez; las siguientes no tocan la red	Medido: 1 petición por sesión
3. Delegación de eventos	3 botones por fila y 10 filas implicaban 30 escuchadores, recreados en cada búsqueda	Un único escuchador en el cuerpo de la tabla identifica el botón pulsado	De 30 escuchadores a 1, sin recrearlos al redibujar
4. Un solo redibujado	Insertar filas una por una obliga a recalcular el diseño en cada vuelta del ciclo	Se arma todo el HTML en una cadena y se asigna una vez	Un recálculo de diseño en lugar de diez

10. Integración del framework · Indicador 9

La mayor parte de la interfaz está construida con los mecanismos propios del framework definido en la asignatura: el sistema de rejilla y las clases de utilidad ordenan las tarjetas de resumen, los campos de filtro, el espaciado y la visibilidad condicional por tamaño de pantalla; el componente de tabla resuelve el listado con encabezado, filas destacables y desplazamiento horizontal; las ventanas modales resuelven el detalle, el formulario y la confirmación, y sus eventos del ciclo de vida se usan para encadenarlas y gestionar el foco; los avisos emergentes entregan la retroalimentación; y los componentes de formulario aportan etiquetas, texto de ayuda y estados de validación. Cada componente propio es un módulo de JavaScript con importaciones y exportaciones explícitas.

La consistencia estructural se mantiene en todo el proyecto: todos los componentes siguen el mismo patrón —un módulo que exporta funciones de dibujo y de conexión de eventos— y ninguno introduce una forma distinta de resolver lo que el framework ya resuelve. Los estilos propios se limitan a la identidad visual del establecimiento y a los ajustes de accesibilidad.

11. Consumo de endpoints · Indicador 10

Todo el consumo pasa por un único módulo, de modo que si cambia una URL o el formato de respuesta se corrige en un solo lugar. Se consumen los seis endpoints exigidos más dos de apoyo.

Método y URL	Propósito	Información enviada	Información recibida
GET /documentos	Listar con filtros y paginación	buscar, tipo, desde, hasta, pagina, limite	Arreglo de documentos y paginación

Método y URL	Propósito	Información enviada	Información recibida
GET /documentos/{id}	Ver o editar un documento	Identificador en la ruta	El documento completo
POST /documentos	Crear	título, tipo, fecha_documento, descripcion, archivo_nombre	Documento creado con su identificador
PUT /documentos/{id}	Actualizar	Identificador y los campos del recurso	Documento actualizado
DELETE /documentos/{id}	Eliminar	Identificador en la ruta	Confirmación e identificador
GET /tipos-documento	Llenar los selectores de tipo	—	Identificador, código y nombre de cada tipo
GET /estadisticas	Tarjetas de resumen	—	Total y cantidad por tipo
GET /salud	Comprobar el servicio antes de pedir datos	—	Estado del servicio

Manejo de estados y errores. Carga: indicador giratorio con la tabla oculta. Sin resultados: mensaje distinto según haya filtros. Validación (400 y 422): los errores se muestran dentro del formulario, campo por campo, más un resumen. Recurso inexistente (404): aviso emergente. Servidor caído: mensaje en lenguaje común con botón de reintentar. También se manejan la respuesta 204 y el JSON con formato inesperado. Una clase de error propia conserva el código HTTP y el arreglo de errores, lo que permite distinguir un error de validación de uno general.

12. Estrategias de seguridad · Indicador 11

Medida	Riesgo que reduce	Ubicación
Escape de HTML en todo dato mostrado	XSS almacenado: un título con código se ejecutaría al pintarlo. Al convertir los caracteres especiales en entidades, se muestra literal	utilidades.js
Validación de los datos ingresados	Datos mal formados: se valida largo, formato de fecha y extensión antes de enviar	formularioDocumento.js
Validación de la extensión del archivo	Registro de archivos ejecutables: solo extensiones ofimáticas y de imagen	config.js
Mensajes de error sin detalle técnico	Divulgación de información útil para un atacante: nunca se muestran trazas ni rutas internas	documentosApi.js
Sin credenciales en el código	Filtración de secretos: todo lo del Frontend viaja al navegador	config.js
Framework servido localmente	Ataques a la cadena de suministro: un CDN comprometido o caído no afecta a la escuela	vendor/
Confirmación en acciones destructivas	Borrado accidental: se muestra el título y el foco parte en «Cancelar»	confirmarEliminar.js
Bloqueo del botón al guardar	Registros duplicados por doble clic o conexión lenta	formularioDocumento.js

Limitación declarada. La aplicación aún no tiene autenticación: quien conozca la dirección puede ver y modificar los documentos. Se declara porque es una limitación real y no un descuido, y porque un Frontend no puede resolverla solo: requiere que el Backend implemente autenticación. Queda como primera prioridad de la retrospectiva.

13. Documentación y difusión · Indicador 12

La documentación se organizó dentro del repositorio, junto al código, para que no se desactualice. El README.md documenta problema y solución, tecnologías, estructura, componentes, instalación, ejecución, endpoints, buenas prácticas, seguridad, optimizaciones, el problema técnico resuelto, las

dificultades y la retrospectiva. Se complementa con tres archivos de evidencia en docs/: la integración verificada contra el Backend, la prueba de accesibilidad con las mediciones de contraste y de teclado, y las optimizaciones con su beneficio medido.

[PENDIENTE OBLIGATORIO: la evaluación exige una breve demostración de la solución, explicando funcionamiento general, componentes principales, integración con el Backend, decisiones técnicas y aprendizajes. Grabe el video o prepare la presentación y registre aquí el enlace.]

14. Retrospectiva y mejora continua · Indicador 13

Al ser un trabajo individual, la priorización no requirió acuerdo entre integrantes sino un criterio explícito: cruzar el impacto para el equipo directivo con el esfuerzo de implementación. Con ese criterio se implementó de inmediato la de mayor impacto y menor esfuerzo. Las de prioridad alta pendientes no dependen solo del Frontend —requieren que el Backend las exponga primero—, por lo que se registran como trabajo de la siguiente iteración y no como omisiones de esta entrega.

#	Situación	Mejora propuesta	Prior.	Responsable	Acción
1	Cualquiera que abra la dirección ve y modifica documentos con datos de apoderados y del personal	Autenticación con control de acceso	Alta	Luis Orellana	Coordinar con el Backend la creación de usuarios y proteger las rutas
2	Solo se registra el nombre del archivo; el documento físico sigue fuera del sistema	Carga real del archivo y descarga desde el detalle	Alta	Luis Orellana	Agregar campo de archivo y consumir el endpoint de carga cuando exista
3	En celular la tabla oculta la descripción y las acciones quedan apretadas	Vista en tarjetas para pantallas pequeñas	Media	Luis Orellana	Variante del componente de listado bajo los 768 px
4	Si se elimina un documento por error no hay forma de recuperarlo	Papelera con eliminación lógica	Media	Luis Orellana	Requiere campo de estado en el Backend y una vista de papelera
5	El botón principal estaba escondido en el menú en celulares	Dejarlo siempre visible	Alta	Luis Orellana	Implementada en esta entrega

Mejora implementada dentro del alcance: la número 5, surgida de la revisión de usabilidad, por ser la de mayor impacto inmediato y menor esfuerzo. También se implementaron las dos correcciones de accesibilidad de la sección 4 y el ajuste de desplazamiento del formulario, todas surgidas de pruebas sobre la aplicación en funcionamiento.

15. Conclusiones

Se construyó la capa Frontend del Gestor Documental cubriendo los trece indicadores: se propuso una estructura de nueve componentes con responsabilidades separadas; se resolvió y documentó un problema técnico real de integración entre ventanas modales; se aplicaron criterios WCAG 2.1 verificados con mediciones; se integraron los componentes contra el Backend real; se documentaron once buenas prácticas; se versionó en un repositorio público; se diseñó una interfaz con las siete operaciones exigidas; se implementaron cuatro optimizaciones, dos con medición; se consumieron ocho endpoints con manejo completo de estados y errores; se aplicaron ocho medidas de seguridad; y se registró una retrospectiva con cinco oportunidades de mejora.

El aprendizaje más valioso fue de criterio: las restricciones del cliente son también restricciones de diseño. Que la escuela cuente con un solo notebook básico determinó que la aplicación no necesite instalación ni dependa de un gestor de paquetes. En lo técnico, el hallazgo más significativo fue comprobar que un defecto puede manifestarse solo cuando el usuario tiene activada una opción de accesibilidad: la accesibilidad no es una capa que se agrega al final, sino una condición que también revela errores de funcionamiento.

16. Declaración de uso de Inteligencia Artificial

DECLARACIÓN EXPRESA: en la elaboración de este proyecto y de este informe SÍ se utilizaron herramientas de inteligencia artificial generativa. Herramienta empleada: Claude, modelo de lenguaje de Anthropic, utilizado mediante la interfaz de línea de comandos Claude Code, que permite al asistente leer y escribir archivos del proyecto y ejecutar comandos en el entorno local.

Tarea	Nivel de uso
Análisis del caso y definición de requerimientos desde la ficha del actor de interés	Trabajo propio; la IA no participó.
Estructura de componentes, código de la interfaz y capa de consumo de la API	Propuesto y generado con apoyo de la IA sobre la estructura definida en la asignatura; revisado y validado por el estudiante.
Guiones de prueba de accesibilidad, contraste y teclado, y redacción del README y de este informe	Generados con apoyo de la IA; resultados y redacción revisados por el estudiante.
Verificación funcional del ciclo CRUD contra el Backend y comprobación de las correcciones	Ejecutada y comprobada por el estudiante.

Justificación técnica. Primero, aceleró tareas repetitivas —documentación y guiones de verificación—, liberando tiempo para el análisis del caso y la comprobación funcional, donde reside el aprendizaje de la unidad. Segundo, sirvió como contraste de las decisiones de diseño, obligando a explicitar los criterios de cada una. Tercero, como apoyo en la depuración: los cuatro defectos corregidos se detectaron ejecutando y midiendo la aplicación, y cada corrección se verificó volviendo a medir. El código generado por una IA no se da por correcto hasta que la comprobación lo confirma.

Alcance y responsabilidad. El estudiante declara que comprende el funcionamiento de la totalidad del código entregado y asume la responsabilidad sobre su contenido. La herramienta fue un instrumento de apoyo y no un sustituto del trabajo propio: la definición del problema, la validación de la solución frente a la necesidad de la escuela y la comprobación funcional se realizaron personalmente.

Nota: *si el uso real fue distinto, ajuste esta sección y borre esta nota.*

17. Referencias

Anthropic. (2025). Claude [modelo de lenguaje de gran escala]. <https://www.anthropic.com/claude>

Bootstrap. (2021). Bootstrap 5.1 documentation. <https://getbootstrap.com/docs/5.1/>

Escuela Básica G-733 Chorombo Bajo. (2025). Ficha VcM actor externo. Comuna de María Pinto.

World Wide Web Consortium. (2018). Web Content Accessibility Guidelines (WCAG) 2.1. <https://www.w3.org/TR/WCAG21/>